# Trabajo Práctico 4 – Ejercicios extra

SISTEMAS OPERATIVOS

# Tracks

**1. Desigualdad de CPU con `nice` (6/10)** **3**

**2. Cuota de CPU con cgroups v2 (7/10)** **4**

**3. Presupuesto de CPU y límite de tiempo (6/10)** **6**

# 1. Desigualdad de CPU con `nice` (6/10)

**TRACK – 6/10**

**Descripción:** medir cómo una preferencia relativa del planificador se convierte en una diferencia de CPU sólo cuando hay tareas competidoras.

El valor de `nice` modifica la preferencia relativa de una tarea de tiempo compartido: un valor menor favorece al proceso y uno mayor lo desfavorece. No es un límite de CPU ni una medición de velocidad. El rango habitual de Linux va de $-20$ a 19; bajar el valor normalmente requiere permisos adicionales.

Escribí un programa o script que lance dos trabajadores CPU-bound idénticos. Cada trabajador debe incrementar un contador durante una cantidad configurable de segundos y comunicar sus incrementos por segundo y su tiempo de CPU. La interfaz debe aceptar una duración con `-seconds N` y la opción `-no-pin`. El programa debe:

- asignar, por defecto, `nice=0` al trabajador favorecido y `nice=19` al otro;
- ejecutar ambos trabajadores sobre una misma CPU, usando afinidad de CPU o una herramienta equivalente;
- informar el valor efectivo de `nice`, los incrementos de cada intervalo y el total de cada trabajador;
- terminar automáticamente después de, por ejemplo, cinco segundos.

Probá la versión con competencia y luego repetila sin fijar la afinidad. Usá el comando correspondiente a tu implementación:

```
$ ./nice_workers --seconds 5
$ ./nice_workers --seconds 5 --no-pin
```

Mientras se ejecuta, podés observar los procesos con `ps` o `top`. Registrá `NI`, `PSR`, `%CPU`, los incrementos por intervalo y el cociente entre los contadores finales. No hace falta usar `NI=-20`: la comparación 0 contra 19 evita depender de permisos administrativos. Si probás un valor negativo, anotá el error y no ejecutes la prueba sobre procesos del sistema.

Algunas preguntas interesantes para considerar:

- Por qué la diferencia es grande al compartir una CPU y casi desaparece al dejar que cada proceso use cualquier CPU disponible?
- El proceso con mayor `nice` siempre recibe menos CPU? Separá la preferencia del resultado observado.
- Qué diferencia hay entre contar iteraciones y medir tiempo de CPU?
- Qué cambia si invertís los valores de `nice` o ejecutás tres trabajadores?

Consultá `man 7 sched` y explicá qué parte de la observación depende de la política normal de tiempo compartido y qué parte depende de la carga creada por el experimento.

## 2. Cuota de CPU con cgroups v2 (7/10)

**TRACK – 7/10**

**Descripción:** aplicar un límite de ancho de banda de CPU a un grupo de procesos y comprobarlo con los contadores del núcleo.

Un *control group* (cgroup) agrupa procesos en una jerarquía. El controlador `cpu` puede repartir tiempo por peso o imponer un límite absoluto. En este track vas a usar cgroups v2 y `cpu.max`; no lo confundas con `nice` ni con `cpu.weight`.

Escribí un programa independiente `cpu_burn.c`. Debe aceptar una duración, ejecutar un loop que consuma CPU, imprimir un contador aproximadamente cada segundo y mostrar al final su tiempo de usuario y de sistema. Compilalo sin optimizar el trabajo fuera del loop:

```
$ gcc -Wall -Wextra -O0 -g cpu_burn.c -o cpu_burn
```

Verificá el entorno antes de modificar nada:

```
$ stat -fc %T /sys/fs/cgroup
$ grep -w cpu /sys/fs/cgroup/cgroup.controllers
```

La primera orden debe indicar `cgroup2fs` y la segunda debe mostrar el controlador `cpu`. Usá un subárbol de cgroup que puedas administrar en tu propia máquina. Si tu distribución usa systemd, podés crear la unidad con `systemd-run --user` y usar el cgroup de esa unidad. No montes ni desmontes cgroups, no muevas procesos del sistema y no modifiques la jerarquía global.

Suponiendo que `$CGROUP_DIR` es un cgroup vacío que podés administrar:

```
$ printf '25000 100000\n' > "$CGROUP_DIR/cpu.max"
$ ./cpu_burn 10 > normal.txt &
$ pid_normal=$!
$ ./cpu_burn 10 > limitado.txt &
$ pid_limitado=$!
$ printf '%s\n' "$pid_limitado" > "$CGROUP_DIR/cgroup.procs"
$ wait "$pid_normal" "$pid_limitado"
$ cat "$CGROUP_DIR/cpu.max"
$ cat "$CGROUP_DIR/cpu.stat"
```

El par `25000 100000` está expresado en microsegundos: el grupo puede usar como máximo 25.000 microsegundos de cada período de 100.000, es decir, aproximadamente un cuarto de una CPU. Repetí el experimento con `50000 100000` y con `max 100000`. Si usás `systemd-run`, el equivalente es `--property=CPUQuota=25%`; localizá el cgroup de la unidad y leé los mismos archivos.

Compará los contadores de ambos programas y registrá `usage_usec`, `nr_periods`, `nr_throttled` y `throttled_usec`. El límite puede producir intervalos desparejos, pero el total de CPU del grupo limitado debe quedar

cerca de su cuota y mostrar eventos de *throttling*. Al terminar, esperá a que el cgroup quede vacío y eliminá sólo el directorio que creaste.

Algunas preguntas interesantes para considerar:

- Por qué `cpu.max` es un límite y `nice` es sólo una preferencia relativa?
- Por qué un grupo con dos procesos comparte la misma cuota?
- Qué significan `nr_throttled` y `throttled_usec`?
- Por qué el límite se expresa como tiempo por período y no como un porcentaje de la máquina completa?
- Qué diferencia observarías si cambiaras `cpu.max` por `cpu.weight`?

Consultá la sección del controlador de CPU de la documentación de cgroups v2 y registrá qué permisos o delegación usa tu entorno.

**BONUS – POR PUNTOS**

Implementá un lanzador `cgroup-run` con la interfaz `cgroup-run -quota PERCENT - COMMAND [ARGS...]`. El porcentaje representa la cuota de una CPU y el período puede ser fijo en 100000 microsegundos.
El lanzador debe:

- crear un cgroup privado en un subárbol que puedas administrar;
- traducir la cuota a `cpu.max` y colocar el comando y sus descendientes en el cgroup antes de que el comando empiece a ejecutar;
- mostrar periódicamente `cpu.stat` mientras el comando está activo;
- manejar terminación normal, error de ejecución y `Ctrl-C`, terminando el grupo de procesos y esperando a que el cgroup quede vacío;
- eliminar sólo el cgroup que creó y devolver el estado del comando.

Probalo con un trabajador CPU-bound, con un shell que lance un descendiente y interrumpiéndolo con `Ctrl-C`. Verificá que la cuota se refleje en `usage_usec` y `nr_throttled`, y que no queden procesos ni directorios luego de cada prueba. Repetí con 25 %, 50 % y 100 %.
Entregá capturas de la cuota, de `cpu.stat` y de la limpieza final, junto con una explicación de cómo evitaste que el comando ejecutara fuera del cgroup.

## 3. Presupuesto de CPU y límite de tiempo (6/10)

**TRACK – 6/10**

**Descripción:** construir un ejecutor acotado que distinga tiempo de CPU de tiempo transcurrido y pueda limpiar un grupo de procesos.

El tiempo de CPU mide cuánto ejecutó una tarea; el tiempo transcurrido también incluye esperas y bloqueos. Escribí `runlimit.c`, un programa con esta interfaz:

```
./runlimit CPU_SECONDS WALL_SECONDS -- COMMAND [ARGS...]
```

El programa debe:

- crear un hijo y hacer que pertenezca a un grupo de procesos propio;
- aplicar en el hijo `setrlimit(RLIMIT_CPU, ...)` antes de `execvp()`;
- usar `clock_gettime(CLOCK_MONOTONIC, ...)` y `wait4(..., WNOHANG, ...)` para controlar el límite de tiempo transcurrido;
- si vence ese límite, enviar `SIGTERM` al grupo completo, esperar un intervalo corto y usar `SIGKILL` sólo para los descendientes que todavía queden;
- informar si el comando terminó normalmente o por una señal, además de su tiempo de usuario, tiempo de sistema y tiempo transcurrido.

Usá un límite blando de CPU igual al argumento y un límite duro un segundo mayor. Así un proceso que no maneje `SIGXCPU` permite observar la señal antes de alcanzar el límite duro. Compilá y probá:

```
$ gcc -Wall -Wextra -O0 -g runlimit.c -o runlimit
$ ./runlimit 1 5 -- sh -c 'while :; do :; done'
$ ./runlimit 5 1 -- sleep 5
$ ./runlimit 5 1 -- sh -c 'sleep 10 & wait'
```

El loop CPU-bound debería alcanzar el límite de CPU antes del límite de pared y terminar por `SIGXCPU`. `sleep` debería consumir casi nada de CPU pero alcanzar el límite de pared. El último caso comprueba que el descendiente no queda ejecutándose cuando termina el shell padre. También probá un comando corto que termine correctamente y verificá que el ejecutor devuelve su código de salida.

Registrá el estado final, las señales, los dos tiempos de CPU y el tiempo transcurrido. No uses comandos sin límite ni pruebes sobre procesos ajenos.

Algunas preguntas interesantes para considerar:

- Por qué el comando que duerme puede superar el tiempo de CPU sin superar el tiempo de pared?
- Qué diferencia hay entre `RLIMIT_CPU` y `cpu.max` de un cgroup?
- Por qué el padre debe controlar un grupo de procesos y no sólo el PID del hijo directo?
- Qué información aportan `WIFEXITED` y `WIFSIGNALED`?
- Cómo se usa `WTERMSIG` junto con `getrusage()`?

- Qué limitación tiene medir el uso de CPU sólo con el proceso padre de un shell que creó descendientes?

Consultá `man 2 getrlimit`, `man 2 wait4` y `man 2 setpgid`; completá con `man 7 signal`.